Building Custom React Hooks

Section 16 — Create, Reuse & Share Hook Logic

A beginner-friendly visual guide — plain English with clear examples

1. Revisiting the Rules of Hooks — & Why Use Hooks?

Hooks are special functions that let you use React features (like state and side effects) inside functional components. They always start with **use** — like useState, useEffect, and your own custom hooks. But there are important rules you must follow!

The 3 Rules of Hooks:

1■■ Rule 1: Only call Hooks at the TOP LEVEL

Never put a hook inside an **if statement**, a **loop**, or a **nested function**. Always call hooks at the very top of your component — before any early returns.

Why? React tracks hooks by the order they are called. If you hide a hook inside an if-block, React loses track and bugs appear.

2■■ Rule 2: Only call Hooks inside React Functions

You can only use hooks inside **functional components** or inside other **custom hooks**. Never call a hook in a plain JavaScript function, a class component, or outside React.

Why? Hooks rely on React's internal engine. Outside of React functions, that engine isn't running — so hooks simply won't work.

3■■ Rule 3: Name custom hooks starting with 'use'

Your custom hook function name **must begin with 'use'** — for example: useFetch, useCounter, useForm, useAuth.

Why? React uses this naming convention to automatically detect hook calls and enforce Rules 1 and 2 above. Without 'use', React won't protect you.

DO vs DON'T — quick example:

■ WRONG

```
function MyComp() {
if (isLoggedIn) {
// Hook inside if = BAD!
const [x, setX] = useState(0);
}
for (let i=0; i<3; i++) {
// Hook in loop = BAD!
useEffect(() => {}, []);
}
}
```

■ CORRECT

```
function MyComp() {
// Hooks always at the top
const [x, setX] = useState(0);
useEffect(() => {}, []);

// Conditions go BELOW hooks
if (!isLoggedIn) {
return <p>Please log in</p>;
}
return <p>Welcome!</p>;
}
```

Why use Hooks (and custom hooks)?

- **Share logic between components** — stop copying the same useState/useEffect code everywhere.
- **Keep components clean and short** — move complex logic out into a dedicated hook file.
- **Easier to test** — hook logic lives in one place and can be tested on its own.
- **No need for class components** — hooks replace lifecycle methods and this.state entirely.

> ***Analogy:*** *Think of hooks like electrical outlets. useState and useEffect are the standard built-in outlets already in your wall. Custom hooks are adaptors you build yourself — they plug into those same outlets but give you a brand new, purpose-built interface for your specific need.*

2. Creating a Custom Hook

A **custom hook** is just a regular JavaScript function whose name starts with **use** and that can call other hooks inside it. That's it! You extract repeated logic from components into this function.

■ Without custom hook

Same fetch logic copied in UserList.jsx AND ProductList.jsx

If the logic changes, you must update it in BOTH places.

Components become long and messy.

■ With custom hook

Logic lives once in useFetch.js

Both components just call useFetch() — one clean line.

Change logic once = fixed everywhere.

Minimal custom hook — useFetch.js:

```
// useFetch.js — our very first custom hook
import { useState, useEffect } from 'react';
```

```
function useFetch(url) { // 'use' prefix is required!
const [data, setData] = useState(null);
const [isLoading, setLoading] = useState(false);
const [error, setError] = useState(null);
useEffect(() => {
setLoading(true);
fetch(url)
.then(res => {
if (!res.ok) throw new Error('Fetch failed');
return res.json();
})
.then(json => setData(json))
.catch(err => setError(err.message))
.finally(() => setLoading(false));
}, [url]);
return { data, isLoading, error }; // Return everything the component needs
}
export default useFetch;
```

3. Custom Hook: Managing State & Returning State Values

Inside a custom hook, you can use any built-in hooks like **useState** and **useEffect** just like inside a component. The key idea: the hook manages its own state, then **returns the values** the component needs — usually as an object or array.

What you can return from a custom hook:

Return format	Example	When to use
Object { }	return { data, isLoading, error }	Multiple values with clear names — most common
Array [ ]	return [value, setValue]	When you want the caller to rename things (like useSta
Single value	return data	Hook only manages one piece of state
Mixed	return { data, reset }	State values + functions together

State is private to each component:

> *Each component that calls your custom hook gets its **own separate copy of the state**. If UserList calls useFetch() and ProductList also calls useFetch(), they each have their own independent data, isLoading, and error — they don't share!*

4. Exposing Nested Functions From The Custom Hook

Custom hooks can also **expose functions** (not just state values) so that the component can trigger actions — like resetting state, re-fetching data, or posting new data. You simply define the function inside the hook and include it in the return value.

Example — useCounter hook that exposes increment, decrement, reset:

```
// useCounter.js
import { useState } from 'react';
function useCounter(initialValue = 0) {
const [count, setCount] = useState(initialValue);
// Define action functions inside the hook
function increment() { setCount(prev => prev + 1); }
function decrement() { setCount(prev => prev - 1); }
function reset() { setCount(initialValue); }
// Return state AND the functions so the component can use them
return { count, increment, decrement, reset };
}
export default useCounter;
```

Using it in a component — super clean:

```
import useCounter from './useCounter';
function ScoreBoard() {
const { count, increment, decrement, reset } = useCounter(0);
return (
Score: {count}
+1
-1
Reset
);
}
```

Key insight: *The component doesn't know or care how the counter works. It just calls increment() and the hook handles everything internally. This is the power of encapsulation — hide the complexity, expose only what's needed.*

5. Using A Custom Hook in Multiple Components

The biggest benefit of custom hooks is **reuse**. Once you write a custom hook, you can use it in as many components as you want. Each component gets its own independent state — they don't interfere with each other.

The useFetch hook used in three different components:

```
// ---- UserList.jsx ----
import useFetch from './useFetch';

function UserList() {
const { data: users, isLoading, error } = useFetch('/api/users');
if (isLoading) return Loading users...;
if (error) return Error: {error};
return {users?.map(u => {u.name})};
}
// ---- ProductList.jsx ----
import useFetch from './useFetch';

function ProductList() {
const { data: products, isLoading } = useFetch('/api/products');
if (isLoading) return Loading products...;
return {products?.map(p => {p.title})};
}
// ---- NewsSection.jsx ----
import useFetch from './useFetch';

function NewsSection() {
const { data: news } = useFetch('/api/news');
return {news?.map(n => {n.headline})};
}
```

UserList	ProductList	NewsSection
`useFetch('/api/users')` `data: [Alice, Bob]` `isLoading: false error: null`	`useFetch('/api/products')` `data: [Chair, Lamp]` `isLoading: true error: null`	`useFetch('/api/news')` `data: null isLoading: false error: '404'`
Own independent state	*Own independent state*	*Own independent state*

6. Creating Flexible Custom Hooks

A flexible custom hook accepts **parameters** so it can be configured differently by each component that uses it. Instead of hardcoding values, you pass them in — making the hook truly reusable across many different situations.

Example — flexible useFetch with options:

```
// useFetch.js — now flexible with parameters
import { useState, useEffect, useCallback } from 'react';
function useFetch(url, options = {}) { // Accept options as second param
const [data, setData] = useState(null);
const [isLoading, setLoading] = useState(false);
const [error, setError] = useState(null);
// fetchData is memoized so it doesn't recreate on every render
const fetchData = useCallback(async () => {
if (!url) return; // Guard: don't fetch if no URL
setLoading(true);
setError(null);
try {
const res = await fetch(url, options); // Pass options to fetch
if (!res.ok) throw new Error(`Error ${res.status}`);
const json = await res.json();
setData(json);
} catch (err) {
setError(err.message);
} finally {
setLoading(false);
}
}, [url]); // Re-run when URL changes
useEffect(() => {
fetchData();
}, [fetchData]);
// Also expose refetch so components can trigger a manual reload
return { data, isLoading, error, refetch: fetchData };
}
export default useFetch;
```

Using the flexible hook in different ways:

```
// Simple GET — just a URL
const { data: users } = useFetch('/api/users');
// With query param — different URL, same hook
const { data: user } = useFetch(`/api/users/${userId}`);
// Manual refetch button
const { data, isLoading, refetch } = useFetch('/api/scores');
// Refresh Scores
// Conditionally fetch — hook guards internally when url is empty
const { data } = useFetch(isLoggedIn ? '/api/profile' : '');
```

Flexibility technique	How to do it	Example
Parameters	Add args to hook function	useFetch(url, method)
Default values	Use = {} or = 'GET'	function useHook(x = 0)

Guard clauses	if (!url) return early	Skip fetch when no URL yet
Expose refetch	Return the fetch function	{ data, refetch }
Dynamic URL	Pass template literal as URL	useFetch(`/api/${id}`)

Quick Reference — Custom Hooks Cheat Sheet

Concept	Key Point
Hook naming rule	Must start with 'use' — e.g. useFetch, useCounter, useForm
Where to call hooks	Only at top level of a component or another custom hook
Create a custom hook	Export a function starting with 'use' that calls built-in hooks
Return state values	return { data, isLoading, error } — object is most flexible
Return functions	Define functions inside the hook and include them in return
State is isolated	Each component gets its own state when it calls the hook
Accept parameters	Add params to make the hook flexible — useFetch(url, options)
Guard against bad input	if (!url) return; — handle empty/null inputs gracefully
Expose refetch/reset	Return the action function so components can trigger it
File convention	Put each hook in its own file: useFetch.js, useCounter.js

> ***Golden Rule:*** *If you find yourself writing the same useState + useEffect pattern in more than one component — that's your signal to extract it into a custom hook!*